```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano 17.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash 17.sh
17.sh: line 9: syntax error near unexpected token `('
17.sh: line 9: `int is_valid_variable_char(char c)'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ cat 17.sh
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_OUTPUT 4096

int is_valid_variable_char(char c)
{
    return isalnum((unsigned char)c) || c == '_';
}

void expand_variables(const char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        /* Detect variable reference */
        if (input[i] == '$')
        {
            i++;

            /* ${VARIABLE} format */
            if (input[i] == '{')
            {
                char variable[256];
                int k = 0;

                i++;

                while (input[i] != '\0' &&
                       input[i] != '}' &&
                       k < 255)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';
```

```
               input[i] != '}' &&
               k < 255)
        {
            variable[k++] = input[i++];
        }

        variable[k] = '\0';

        if (input[i] == '}')
            i++;

        char *value = getenv(variable);

        if (value != NULL)
        {
            while (*value != '\0' &&
                   j < MAX_OUTPUT - 1)
            {
                output[j++] = *value++;
            }
        }
        else
        {
            printf("Undefined variable: %s\n",
                   variable);
        }
    }

    /* $VARIABLE format */
    else if (isalpha((unsigned char)input[i]) ||
             input[i] == '_')
    {
        char variable[256];
        int k = 0;

        while (is_valid_variable_char(input[i]) &&
               k < 255)
        {
            variable[k++] = input[i++];
        }

        variable[k] = '\0';

        char *value = getenv(variable);

        if (value != NULL)
        {
```

```
                int k = 0;

                while (is_valid_variable_char(input[i]) &&
                       k < 255)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                char *value = getenv(variable);

                if (value != NULL)
                {
                    while (*value != '\0' &&
                           j < MAX_OUTPUT - 1)
                    {
                        output[j++] = *value++;
                    }
                }
                else
                {
                    printf("Undefined variable: %s\n",
                           variable);
                }
            }
            else
            {
                output[j++] = '$';
            }
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("=========================================\n");
    printf("       VARIABLE EXPANSION\n");
```

26°C

```
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("=========================================\n");
    printf("       VARIABLE EXPANSION\n");
    printf("=========================================\n");

    printf("\nSet test variables:\n");

    setenv("NAME", "Shanmukha", 1);
    setenv("COURSE", "BTech", 1);
    setenv("YEAR", "2", 1);

    printf("NAME   = %s\n", getenv("NAME"));
    printf("COURSE = %s\n", getenv("COURSE"));
    printf("YEAR   = %s\n", getenv("YEAR"));

    printf("\nEnter a string containing variables:\n");
    printf("Example: Hello $NAME, you are studying $COURSE\n\n");

    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("\nOriginal:\n");
    printf("%s\n", input);

    printf("\nExpanded:\n");
    printf("%s\n", output);

    printf("\nNested variable test:\n");

    setenv("USER_NAME", "NAME", 1);
```

```
    printf("NAME   = %s\n", getenv("NAME"));
    printf("COURSE = %s\n", getenv("COURSE"));
    printf("YEAR   = %s\n", getenv("YEAR"));

    printf("\nEnter a string containing variables:\n");
    printf("Example: Hello $NAME, you are studying $COURSE\n\n");

    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("\nOriginal:\n");
    printf("%s\n", input);

    printf("\nExpanded:\n");
    printf("%s\n", output);

    printf("\nNested variable test:\n");

    setenv("USER_NAME", "NAME", 1);

    char nested[256];

    strcpy(nested, "$USER_NAME");

    expand_variables(nested, output);

    printf("Input  : %s\n", nested);
    printf("Output : %s\n", output);

    printf("\nVariable expansion completed.\n");

    return 0;
}
```

```c
#define MAX_ARGS 64

typedef void (*builtin_function)(char **args);

void builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
    }
}

void builtin_pwd(char **args)
{
    char directory[1024];

    if (getcwd(directory, sizeof(directory)) != NULL)
    {
        printf("%s\n", directory);
    }
    else
    {
        perror("pwd");
    }
}

void builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");

        i++;
    }

    printf("\n");
```

```
}

void builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");

        i++;
    }

    printf("\n");
}

void builtin_set(char **args)
{
    if (args[1] == NULL || args[2] == NULL)
    {
        printf("Usage: set <variable> <value>\n");
        return;
    }

    if (setenv(args[1], args[2], 1) != 0)
    {
        perror("set");
        return;
    }

    printf("%s=%s\n", args[1], args[2]);
}

void builtin_get(char **args)
{
    char *value;

    if (args[1] == NULL)
    {
        printf("Usage: get <variable>\n");
        return;
    }

    value = getenv(args[1]);
```

```
        return;
    }

    value = getenv(args[1]);

    if (value != NULL)
        printf("%s\n", value);
    else
        printf("Variable '%s' is not set.\n",
               args[1]);
}

void builtin_help(char **args)
{
    printf("\nAvailable built-in commands:\n");
    printf("  cd <directory>     Change directory\n");
    printf("  pwd                Print current directory\n");
    printf("  echo <text>        Print text\n");
    printf("  set <var> <value>  Set environment variable\n");
    printf("  get <var>          Get environment variable\n");
    printf("  help               Show this help\n");
    printf("  exit               Exit shell\n");
}

void builtin_exit(char **args)
{
    exit(0);
}

struct builtin
{
    const char *name;
    builtin_function function;
};

struct builtin dispatch_table[] =
{
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"echo", builtin_echo},
    {"set", builtin_set},
    {"get", builtin_get},
    {"help", builtin_help},
    {"exit", builtin_exit}
};
```

```
int builtin_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);

int find_builtin(const char *command)
{
    int i;

    for (i = 0; i < builtin_count; i++)
    {
        if (strcmp(command, dispatch_table[i].name) == 0)
            return i;
    }

    return -1;
}

int parse_command(char *input, char **args)
{
    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;

        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    return count;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=========================================\n");
    printf("       BUILT-IN COMMAND DISPATCHER\n");
    printf("=========================================\n");

    printf("\nType 'help' for available commands.\n");
```

```c
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("==========================================\n");
    printf("       BUILT-IN COMMAND DISPATCHER\n");
    printf("==========================================\n");

    printf("\nType 'help' for available commands.\n");

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (input[0] == '\n')
            continue;

        parse_command(input, args);

        if (args[0] == NULL)
            continue;

        int index = find_builtin(args[0]);

        if (index == -1)
        {
            printf("Invalid command: %s\n",
                   args[0]);

            printf("Type 'help' to see available commands.\n");
            continue;
        }

        dispatch_table[index].function(args);
    }

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```